

Univerzita Karlova

Přírodovědecká fakulta

Studijní program: Geoinformatika, kartografie a dálkový průzkum Země



Aliaksandra Sparysh

Jolana Václavková

Geoinformatika

Prostorová indexace

1. Úvod

Cílem této úlohy bylo implementovat prostorovou indexaci a následně pomocí ní určit dvě základní charakteristiky zadaného bodového mračna, a to prostorovou hustotu a křivost v jednotlivých bodech.

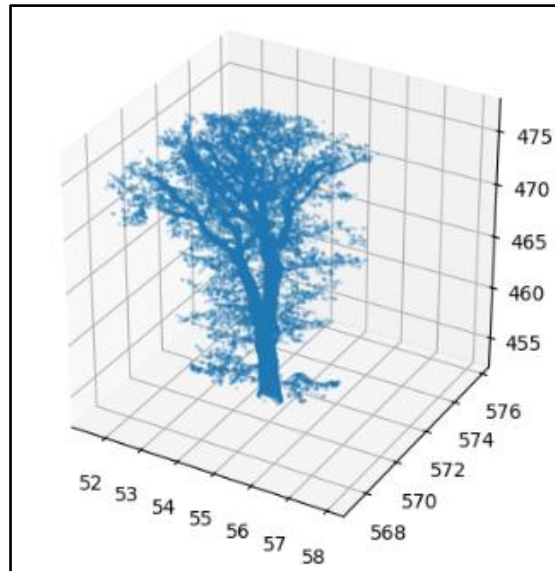
Prostorová hustota popisuje míru koncentrace bodů v okolí daného bodu. Vyjadřuje, jak velký objem zaujímá nejbližší sousedství. Hustota tak slouží jako lokální parametr charakterizující strukturu bodového mračna. Křivost popisuje, jak moc se tvar v okolí bodu odchyluje od roviny. Pokud je nejmenší vlastní číslo kovarianční matice malé, oblast je téměř rovinná, větší odchylka znamená výraznější zakřivení.

Pro prohledávání bodového mračna byly použity metody A). naivní hledání, B). akceleroaná voxelizace, C). kd-tree a D). Octree. Zadání předpokládá, že všechny vyhledávací metody jsou implementovány samostatně, bez využití knihoven pro práci s prostorovými daty. V textu je popsána implementace jednotlivých metod, jejich výhody a nevýhody, použití při výpočtu hustoty i křivosti a také vizualizace výsledků. Součástí je rovněž porovnání výpočetního času jako funkce velikosti datové množiny a u metody voxelizace také jako funkce velikosti voxelu.

2. Zadané bodové mračno

Vstupní dataset tvoří bodové mračno reprezentující tvar skutečného stromu (Obr. 1). Soubor obsahuje celkem 25 737 bodů (získáno voláním funkce `loadPoints`, Obr. 2, z nichž každý je definován trojicí souřadnic (X,Y,Z). Body zachycují jak kmen, tak korunu stromu, včetně větví.

Obr. 1: Zadané bodové mračno



Obr. 2: Počet bodů volaný pomocí funkce `loadPoints`

```
x, y, z = loadPoints('tree_18.txt')
'Number of points:', len(x)

('Number of points:', 25737)
```

3. Metoda naivního hledání pomocí nejbližšího souseda

Metoda nejbližšího souseda (NN) funguje tak, že pro každý bod v mračnu postupně porovná jeho vzdálenost ke všem ostatním bodům, aby našla ten nejbližší.

Funkce `getDensityNN` odhaduje prostorovou hustotu mračna bodů pomocí naivního vyhledávání nejbližšího souseda pro každý bod. Na vstupu dostává tři pole souřadnic (X, Y, Z). Pro každý bod *i* pak postupně prochází všechny ostatní body v datasetu, u každého spočítá eukleidovskou vzdálenost a průběžně si ukládá nejmenší nalezenou hodnotu. Tím je zaručeno,

že po skončení cyklu je pro bod i známá vzdálenost k jeho skutečně nejbližšímu sousedovi (s výjimkou samotného bodu). Tyto nejmenší vzdálenosti se uloží do pole `distances`.

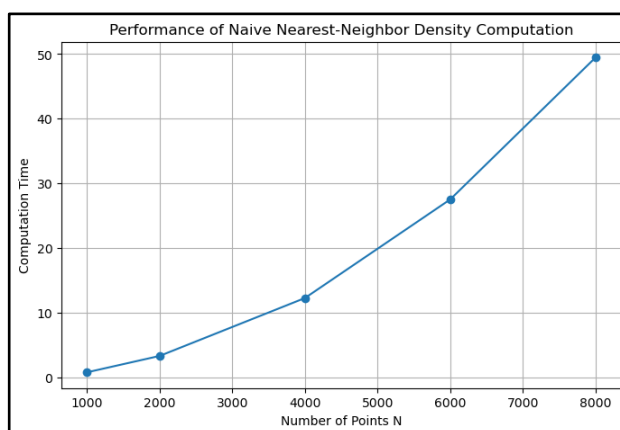
Po projití všech bodů je spočítána průměrná hodnota těchto nejbližších vzdáleností a hustota se určí jako převrácená hodnota třetí mocniny tohoto průměru (viz Obr. 3).

Obr. 3: Hustota zadaného bodového mračka metodou naivního hledání pomocí nejbližšího souseda

<code>'Density using nearest neighbor distance (NN):', getDensityNN(X, Y, Z)</code>
Density using nearest neighbor distance (NN): 6060.374340762823

Výpočet je náročný, protože pro každý bod se porovnávají vzdálenosti ke všem ostatním bodům, což je pro velká mračka (v našem případě přes 25 tisíc bodů) výpočetně velmi pomalé (Obr. 4).

Obr. 4: Výpočetní čas metody naivního hledání pomocí nejbližšího souseda v závislosti na počtu bodů



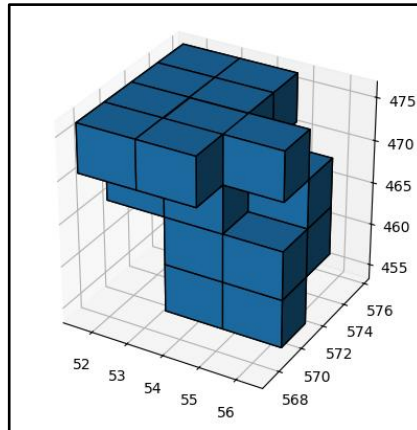
4. Akcelerované hledání s využitím voxelizace

Princip metody spočívá v tom, že celý prostor obsahující bodové mračko je rozdělen na pravidelnou trojrozměrnou mřížku (voxely). Každý bod je podle svých souřadnic přiřazen do jednoho voxelu, čímž získáme přehled o tom, které body jsou si v prostoru blízko.

Aby bylo možné určit celkový rozsah bodového mračka, jsou nejprve spočítány minimální a maximální hodnoty souřadnic X , Y a Z . Tento rozsah se rovnoměrně rozdělí na zvolený počet voxelů. Každý bod pak podle svých souřadnic získá informaci o tom, do kterého voxelu patří. Poté se každý bod uloží do hash tabulky `hashTable`. Klíčem je číselná adresa voxelu (jedno

číslo, které reprezentuje jeho pozici v trojrozměrné mřížce) a hodnotou je seznam indexů bodů, které do daného voxelu spadají.

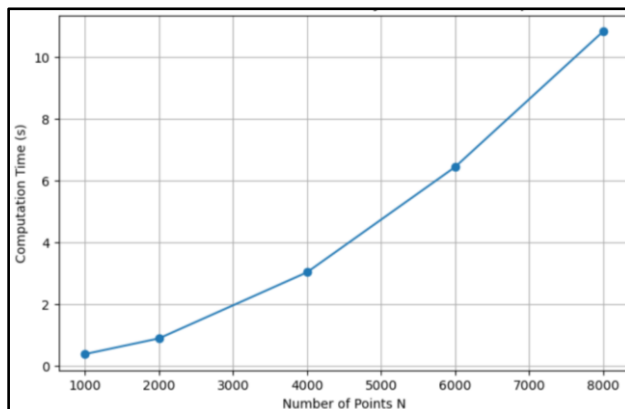
Obr. 5: Reprezentace bodového mračna pomocí voxelů



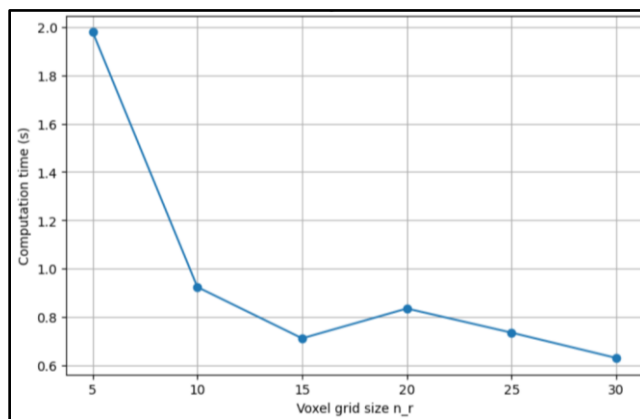
Při hledání nejbližšího souseda pro daný bod se nejprve prohledá voxel, ve kterém se bod nachází. Pokud je tento voxel prázdný nebo obsahuje pouze samotný bod, rozšíří se vyhledávání do všech 26 sousedních voxelů. Pro všechny body z těchto voxelů se spočítá kvadratická eukleidovská vzdálenost a vybere se nejmenší z nich. Tímto způsobem funguje funkce `getNNForPoint`, která vrací index nejbližšího souseda a jeho vzdálenost. Výpočet hustoty `getDensity` pak pro každý bod v datasetu zavolá právě tuto metodu. Výsledkem je seznam vzdáleností k nejbližším sousedům, ze kterého se vypočítá jejich průměrná hodnota. Hustota se potom získá jako převrácená hodnota třetí mocniny tohoto průměru (čím menší je průměrná vzdálenost mezi body, tím vyšší je výsledná hustota).

Výhodou tohoto přístupu je výrazné zrychlení oproti naivní metodě, protože prohledává pouze omezený počet bodů uložených ve stejném nebo sousedních voxelech. V grafu (Obr.6) je dobře vidět, že čas výpočtu roste mnohem pomaleji než u naivní metody (Obr.4). Nevýhodou je, že přesnost metody závisí na velikosti voxelové mřížky. Pokud jsou voxely příliš velké, skončí v jednom voxelu mnoho bodů a metoda ztrácí rychlost i přesnost. Pokud jsou naopak příliš malé, nejbližší sousedi mohou být mimo oblast, kterou algoritmus kontroluje.

Obr. 6: Výpočetní čas metody akcelerovaného hledání s využitím voxelizace v závislosti na počtu bodů



Obr. 7: Výpočetní čas v závislosti na velikosti voxelové mřížky



Pozn: Závislost výpočetního času na velikosti voxelové mřížky není zcela monotónní. Nepravidelnosti můžou být způsobené náhodným rozdělením bodů do voxelů či šumem měření.

Křivost se v této metodě odhaduje pomocí PCA (Principal component analysis) z lokálního okolí každého bodu. Pro každý bod se vybere 30 jeho nejbližších sousedů a z jejich prostorového rozmístění se určí, jak moc se toto lokální okolí odchyluje od roviny. Tato míra odchylky pak slouží jako odhad lokální křivosti.

To se provádí analýzou variability sousedních bodů ve třech hlavních směrech. Pokud má bod sousedů málo, nastaví se jeho křivost na nulu, aby se předešlo chybám ve výpočtu:

```
if len(neighbors) < 3: curvature[i] = 0.0
```

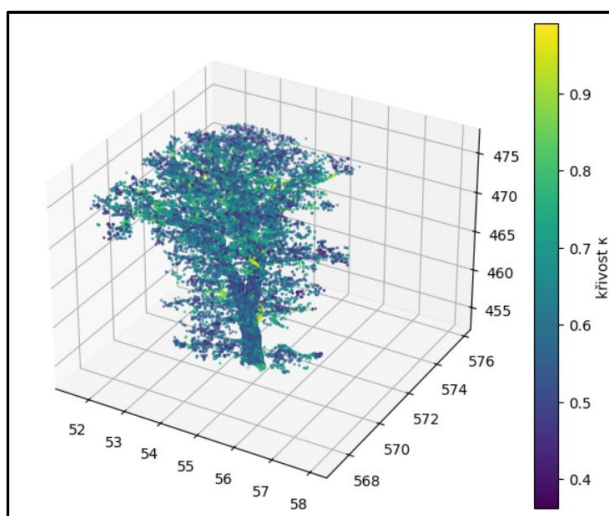
Výsledná křivost byla spočítána jako poměr variability ve směru, který nejvíce vystupuje nad ostatní dva. Pokud tento poměr dominuje, okolí bodu je výrazně zakřivené, pokud jsou směry variabilit podobné, křivost je malá.

Výsledné hodnoty a vizualizace křivosti jsou zobrazeny na Obr. 8 a Obr. 9.

Obr. 8: Hustota a křivost zadaného bodového mračka metodou akcelerovaného hledání pomocí voxelizace

```
'Density using approximate nearest neighbor distance (ANN) with voxels:', hashTable.getDensity()  
hashTableCurvs = hashTable.getCurvatures()  
'Curvature for all points', hashTableCurvs  
  
Density using approximate nearest neighbor distance (ANN) with voxels: 5507.542382287587  
Curvature for all points [0.63761835 0.56797931 0.65517465 ... 0.4741361 0.44202822 0.50474263]
```

Obr. 9: Vizualizace křivosti pomocí akcelerovaného hledání s využitím voxelizace



5. Akcelerované hledání s využitím kd-tree

Další akcelerační metodou použitou při vyhledávání nejbližších sousedů je KD-tree (k-dimenzionální strom). Na rozdíl od voxelizace, kde je prostor dopředu pevně rozdělen, se KD-tree přizpůsobuje skutečnému rozmístění bodů v datasetu. Každý uzel stromu (`KDNode`) obsahuje souřadnice jednoho bodu, informaci o ose dělení (`axis`) a ukazatele na levého a pravého potomka. Třída `KDTree` pak slouží jako hlavní obal celé struktury a uchovává vstupní data i kořen stromu.

Strom se buduje rekurzivně. Na začátku se vezme celý seznam bodů a zvolí se souřadnicová osa, podle které se budou body rozdělovat. Indexy bodů se tedy seřadí podle osy a medián se zvolí jako uzel. Body s menší hodnotou jdou do levé větve, větší do pravé větve. Z bodů na levé a pravé straně mediánu se opět vytvoří menší podstromy. Tento postup se opakuje, dokud nejsou rozděleny všechny body.

Při vyhledávání nejbližšího souseda v KD-tree se využívá rekurzivní metoda `getNN`. Pro daný bod se nejprve načtou jeho souřadnice a prohledávání začne v kořeni stromu. V každém uzlu se spočítá kvadratická eukleidovská vzdálenost mezi dotazovaným bodem a bodem uloženým v uzlu. Pokud je tato vzdálenost menší než dosud nejlepší nalezená hodnota, uloží se jako nové minimum a aktuální uzel se “zapamatuje” jako kandidát na nejbližšího souseda. Podle dělicí osy se pak rozhodne, do které části stromu má algoritmus jít nejdříve – pokud má dotazovaný bod na této ose menší hodnotu než bod v uzlu, pokračuje se do levé větve, v opačném případě do pravé. Teprve po prohledání bližší větve se zvažuje, jestli má smysl vstoupit i do druhé, vzdálenější, větve. To se však provede jen tehdy, pokud vzdálenost bodu od dělicí roviny může být stále menší než dosud nejlepší nalezená vzdálenost.

Pro výpočet hustoty v `getDensity` se zavolá funkce `getNNForPoint`, která použije vyhledávání `getNN` a vrátí vzdálenost k nejbližšímu sousedovi. Tyto vzdálenosti se uloží do pole, a spočítá se z nich průměrná hodnota, hustota se určí podle již výše použitého vzorce:

$$\text{density} = 1 / (\text{avg_distance} ** 3)$$

Pro výpočet křivosti byla využita metoda `getKNNForPoint`, která v KD-tree rekurzivně vyhledá body nejbližší dotazovanému bodu a ukládá je tak, aby vždy zůstalo právě 30 nejbližších sousedů. Jakmile jsou nalezeni, jejich souřadnice se převedou do matice, kde se vypočítá jejich těžiště. Všechny body okolí se od tohoto středu „odpočítají“, aby se pracovalo pouze s jejich vzájemnou polohou. Z takto vycentrovaných bodů se vytvoří kovarianční matice, která popisuje, jak moc se sousedé rozkládají v jednotlivých směrech prostoru. Následně se konstruují její vlastní čísla – tři hodnoty, které vyjadřují variabilitu bodů ve třech hlavních směrech. Křivost je poté určena jako podíl nejmenšího vlastního čísla k součtu všech tří.

$$\text{kappa} = \text{lambdas}[0] / \text{lam_sum}$$

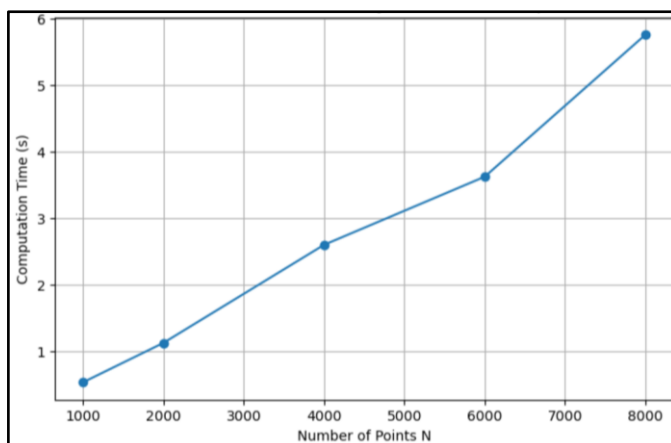
Tento postup se pomocí funkce `getCurvatures` zopakuje pro všechny body v mračnu, takže výsledkem je pole hodnot, zachycující vlastnosti celého tvaru stromu, vizualizováno na (Obr.12).

Obr. 10: Hustota a křivost zadaného bodového mračna metodou akcelerovaného hledání s využitím kd-tree

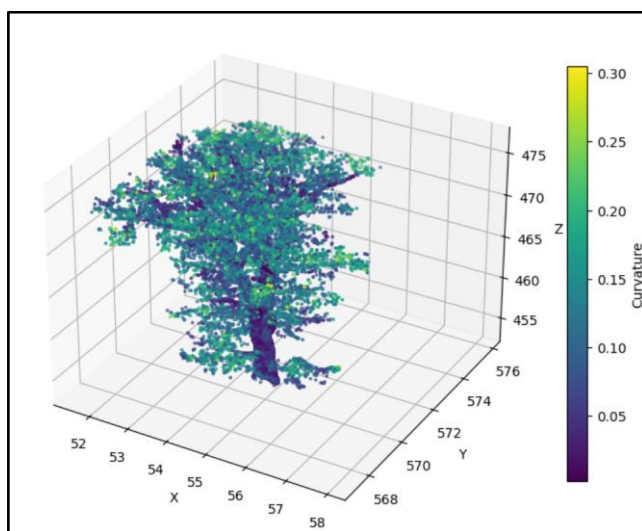
```
'Density using K-D tree:', kdTree.getDensity()  
'Curvature of every point using KD tree:', kdTree.getCurvatures()  
  
Density using K-D tree: 6060.374340762823  
Curvature of every point using KD tree: [0.02019061 0.01370536 0.02703648 ... 0.14296777 0.13848764 0.1177637 ]
```

KD-tree je rychlejší než naivní metoda (Obr.11) a zároveň poskytuje přesné výsledky, protože nejbližší sousedy skutečně dohledá. Strom se navíc přirozeně přizpůsobuje rozmístění dat, takže není nutné volit žádné další parametry, jako je tomu u voxelizace. Na druhou stranu je implementace KD-tree složitější než u jednodušších přístupů, u malých množin dat pak mohou být jednodušší metody rychlejší.

Obr. 11: Výpočetní čas metody akcelerovaného hledání s využitím kd-tree v závislosti na počtu bodů



Obr. 12: Vizualizace křivosti pomocí akcelerovaného hledání s využitím kd-tree



6. Akcelerované hledání s využitím Octree

Metoda využívající Octree rozděluje prostor bodového mračka na hierarchickou strukturu krychlí, které se postupně dělí podle toho, kolik bodů obsahují. Při vyhledávání nejbližších sousedů se tedy algoritmus soustředí jen na ty části prostoru, kde se soused skutečně může nacházet, a ignoruje vzdálené oblasti.

Vytvořená implementace nejprve konstruuje Octree nad vstupním bodovým mračnem. Každý uzel uchovává seznam indexů bodů a vlastní osově orientovaný ohraničující kvádr. Pokud uzel překročí povolený počet bodů (v našem případě nastaveno na hodnotu 100) a zároveň není dosažena maximální povolená hloubka stromu (v našem případě hodnota 10), je jeho prostor rozdělen na osm menších kvádrů a body jsou do nich znovu roztrženy podle své polohy. Tím vzniká hierarchická struktura, která umožňuje efektivně omezit počet porovnávaných bodů během vyhledávání nejbližších sousedů.

Vyhledávání nejbližšího souseda využívá rekursivní postup, jenž prochází nejprve ten oktant, ve kterém se dotazovaný bod nachází. Teprve následně jsou kontrolovány další uzly — a to pouze ty, u nichž minimální možná vzdálenost k ohraničujícímu kvádru není větší než aktuálně nejlepší nalezená vzdálenost. Výpočet samotné hustoty bodového mračka je již založen na obdobném principu jako předešlé metody - získané vzdálenosti se zprůměrují a výsledná hustota se určí jako hodnota převrácená hodnota třetí mocniny tohoto průměru.

Výpočet křivosti vychází z předpokladu, že lokální vlastnosti lze odhadnout z polohy nejbližších bodů v okolí. Prvním krokem je proto nalezení 30 nejbližších sousedů každého

bodu. K tomu slouží funkce `query_knn`. Pomocí rekurzivního procházení (`collect_indices`) jsou vyhledávány pouze ty uzly, které ještě mohou obsahovat body bližší než dosud nalezený kandidát. To znamená, že celé části stromu se přeskočí, pokud je zřejmé, že jejich ohraničující kvádr leží příliš daleko. Tímto způsobem se průběžně shromažďují potenciálně blízké body, dokud není shromážděna množina 30 nejbližších sousedů.

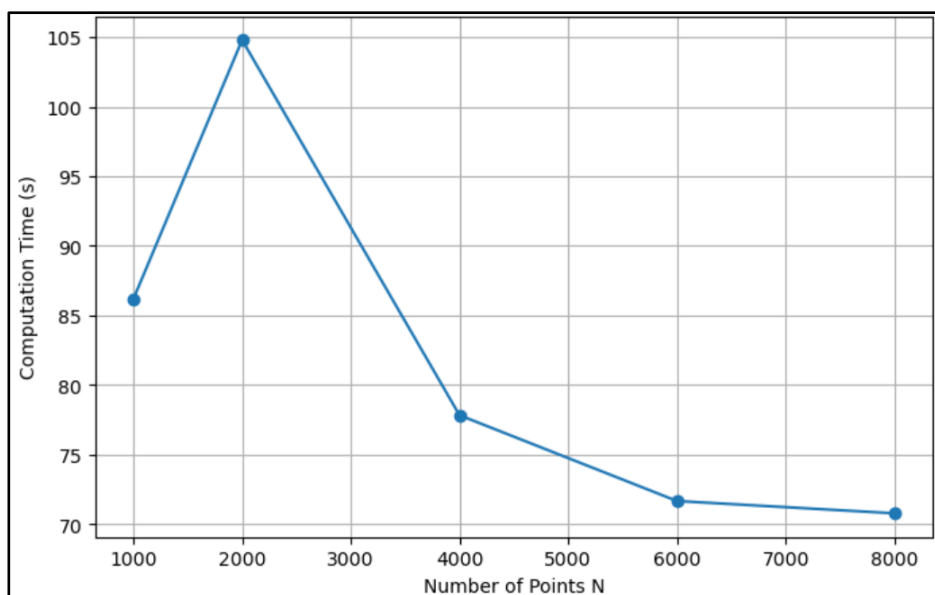
Samotný výpočet křivosti probíhá funkcí `curvature_from_neighbors`, která, podobně jako v předchozích implementacích, využívá metodu hlavních komponent (PCA). Sousedi jsou nejprve centrováni vůči svému průměru a z jejich rozložení je vytvořena kovarianční matice. Z jejich vlastních hodnot se následně získá poměr nejmenší vlastní hodnoty k součtu všech tří, který slouží jako míra lokální křivosti. Výsledné hodnoty jsou vidět na (Obr. 13).

Obr. 13: Hustota a křivost zadaného bodového mračka metodou akcelerozaného hledání s využitím Octree

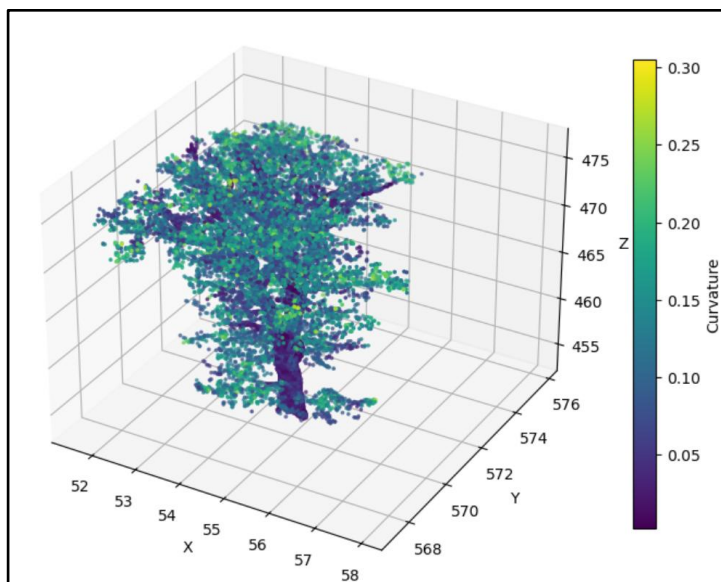
```
'Density using Octree:', octree.getDensity()  
(  
  'Density using Octree:', np.float64(6060.374340762823))  
  
'Curvature using Octree:', octree.getCurvatures()  
(  
  'Curvature using Octree:',  
  array([0.02019061, 0.01370536, 0.02703648, ..., 0.14296777, 0.13848764,  
         0.1177637 ], shape=(25737,)))
```

Jak lze vidět na Obr. 14, naměřené výpočetní časy metody akcelerozaného vyhledávání s využitím Octree vykazují výrazně nevyrovnaný průběh. Tento průběh je způsoben tím, že efektivita octree závisí především na geometrické distribuci bodů v prostoru, nikoli pouze na jejich počtu. U menších souborů může být strom hůře vyvážený nebo obsahovat málo obsazené větve, takže hledání nejbližších sousedů vyžaduje více rekurzivních sestupů i kontrol sousedních uzlů. Naopak u větších dat se strom zaplní rovnoměrněji, dělicí roviny lépe odrážejí skutečné prostorové rozložení a vyhledávání probíhá ve výrazně menším počtu větví, což snižuje celkový výpočetní čas. Samotná vizualizace je zobrazena na Obr. 15.

Obr. 14: Výpočetní čas metody akcelerovaného hledání s využitím Octree



Obr. 15: Vizualizace křivosti pomocí akcelerovaného hledání s využitím Octree



7. Závěr

Na zadaném bodovém mračnu stromu byly postupně porovnávány čtyři různé přístupy k prostorové indexaci – naivní vyhledávání, voxelizaci, KD-tree a Octree. Naivní metoda je jednoduchá na implementaci a dává přesné výsledky, ale kvůli kvadratické časové složitosti je pro větší počty bodů velmi pomalá. Voxelizace výrazně zrychlila hledání tím, že omezila vyhledávání pouze na body ve stejném nebo sousedních voxelech, za cenu určité aproximace dané volbou velikosti mřížky. KD-tree se ukázalo jako dobrý kompromis mezi rychlostí a přesností – strom se přizpůsobuje rozmístění bodů a nevyžaduje volbu velikosti voxelu. Octree lépe respektuje 3D charakter dat a v principech nabízí podobné výhody, jeho výkon ale výrazně závisí na rozložení bodů a zvolených parametrech (maximální počet bodů v uzlu a hloubka stromu).